SIMATS ENGINEERING

CO 3 - ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks **COURSE CODE:** CSA0706

COURSE OUTCOME COVERED

CO3: *Analyse the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Faculty: Dr. V. Parthipan

QUESTIONS:4 Total Marks: 40

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M Nishanth**

Criteria	Understanding the case 2.5	**Application of Concepts** 2.5	**Analysis** 2	**Solution Design** 2	**Clarity** 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

QUESTIONS:4 Total Marks:40

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of realtime multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services.

Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyse how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4. A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyse three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

SIMATS ENGINEERING

CO3 – ASSESSMENT TOOL 2: CASE STUDY

Question 1: Video Conferencing Platform – Voice/Video Quality Degradation

Case Recap

A multinational company's cloud video conferencing platform shows voice distortion, frozen frames and delayed communication during peak hours, with 8% packet loss, 180 ms jitter and fluctuating bandwidth.

1. Is the problem caused by the Transport Protocol or the Application Layer?

The symptoms described — packet loss, jitter, and delay leading to frozen frames and distorted audio — point primarily to the transport layer, specifically the protocol used to carry real-time media. If the platform is using TCP for audio/video streams, TCP's reliability mechanisms (retransmission, in-order delivery, and congestion-triggered flow control) directly explain the observed symptoms: a lost packet forces TCP to pause delivery of all subsequent data until the missing segment is retransmitted and acknowledged, which manifests as frozen video frames and rising jitter under 8% loss. Congestion control (slow start / AIMD) also causes the fluctuating bandwidth utilization noted in the report.

However, application-level buffering strategy is a contributing secondary cause. Even where UDP is used correctly for the media stream, an oversized or poorly tuned jitter buffer at the application layer can convert transient network jitter into perceptible delay, or a jitter buffer that is too small can drop late-arriving packets, producing the same distortion symptoms. Therefore, the analysis should conclude that the root cause is transport-protocol behaviour (TCP being unsuitable for loss-tolerant, delay-sensitive real-time media), compounded by an application-layer buffering strategy that is not adapted to the network conditions.

2. Justification Based on Real-Time Multimedia Communication Requirements

Real-time multimedia applications such as video conferencing have fundamentally different requirements from traditional data transfer:

- Low, bounded latency is more important than perfect reliability — humans tolerate an occasional dropped frame far better than a multi-second freeze.

- Timeliness over completeness — a video frame that arrives late is useless even if it eventually arrives correctly, whereas TCP guarantees correctness at the cost of timeliness.
- Continuous stream delivery — TCP's head-of-line blocking (waiting for a retransmitted segment before delivering later data to the application) is incompatible with a continuous, real-time audio/video stream.
- Tolerance to loss — codecs such as Opus (audio) and H.264/VP8 (video) are designed with built-in error concealment and can gracefully degrade quality with a few lost packets, but cannot tolerate the stalls introduced by TCP retransmission.

Because UDP does not perform retransmission or enforce ordering, it allows the application to keep the media stream flowing in near real time, discarding or concealing occasional lost packets instead of stalling. This is why virtually all standard real-time conferencing systems carry media over UDP (typically via RTP/RTCP), reserving TCP only for signalling (e.g., SIP/WebSocket control channels) where reliability, not timeliness, is paramount. This directly justifies the diagnosis in part 1: the 8% loss and 180 ms jitter are within the range that RTP-over-UDP with proper concealment can handle gracefully, but the same conditions cause severe degradation if TCP is being used for media.

3. Recommended Protocol-Level and Application-Level Improvements

Protocol-level improvements:

- Migrate real-time audio/video streams to UDP-based RTP/RTCP (or SRTP for security) instead of TCP, reserving TCP/WebSocket only for call-signalling.
- Use RTCP feedback and adaptive bitrate/codec switching (e.g., WebRTC's built-in congestion control, such as Google Congestion Control) to react to changing bandwidth and packet loss in real time.
- Enable Forward Error Correction (FEC) and Packet Loss Concealment (PLC) at the codec level so isolated losses do not produce audible/visible artefacts.
- Apply Quality of Service (QoS) marking (DSCP/EF class) on the network so voice/video traffic is prioritized over best-effort traffic during peak congestion.

Application-level improvements:

- Implement an adaptive jitter buffer that dynamically resizes based on measured network jitter instead of using a fixed buffer size.
- Use simulcast/SVC (Scalable Video Coding) so the server can drop to a lower resolution/frame-rate layer for congested clients without renegotiating the whole call.
- Deploy edge media relay servers (SFU/TURN servers) closer to major client clusters to reduce round-trip time and the probability of loss on long paths.
- Continuously monitor call quality metrics (MOS score, jitter, loss, RTT) and trigger automatic bandwidth/resolution adaptation before quality visibly degrades.

Question 2: Redesigning DNS Infrastructure for a Global Web Application

Case Recap

A globally hosted web application experiences an average DNS lookup time of about 800 ms, and the organization wants to redesign its DNS infrastructure to bring lookup time below 50 ms while maintaining availability during server failures.

1. Reasons Behind the High DNS Resolution Time

- Centralized or geographically distant authoritative DNS servers: if all queries are resolved by a single server (or a small set of servers) located in one region, users on other continents must traverse long round-trip paths, adding significant latency.
- Lack of caching / low TTL values: if the Time-To-Live (TTL) on DNS records is set too low, resolvers must repeatedly query the authoritative server instead of serving cached responses, multiplying the cost of every network round trip.
- Deep recursive resolution chains: multiple hops through root, TLD, and authoritative name servers (recursive resolution) add cumulative latency, especially without local caching resolvers.
- No use of Anycast routing: without Anycast, all client requests converge on one physical server location rather than being routed to the topologically nearest server.
- Server-side bottlenecks: an overloaded or under-provisioned DNS server experiencing high query volume during peak periods adds queuing delay on top of network latency.

- Absence of DNS pre-fetching in client applications, causing lookups to occur synchronously on the critical path of a user's first request instead of being resolved in advance.

2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

A redesigned, latency-optimized DNS architecture should combine the following techniques:

- Anycast DNS: Deploy the same authoritative DNS IP address from multiple Points of Presence (PoPs) around the world. Internet routing (BGP) automatically directs each user's query to the topologically/geographically nearest DNS server, cutting round-trip time dramatically without any client-side configuration.
- TTL Optimization: Set TTL values appropriately — long enough (e.g., 3,600–86,400 seconds for stable records) to maximize cache hits at recursive resolvers and reduce repeated authoritative lookups, while keeping TTLs short enough (e.g., 60–300 seconds) only for records that must support fast failover, such as those behind a load balancer.
- DNS Pre-fetching: Use browser/application-level DNS pre-fetching (e.g., HTML <link rel="dns-prefetch">) and OS-level resolver pre-fetching so that domain names likely to be needed are resolved in the background before the user actually requests them, removing the lookup from the critical path entirely.
- Layered caching: Deploy caching resolvers close to users (ISP-level and CDN edge resolvers) so that the majority of queries are answered from cache rather than reaching the authoritative servers at all.
- Redundant, geographically distributed authoritative name servers with health checks and automatic failover, so that if one PoP fails, Anycast routing seamlessly redirects traffic to the next-nearest healthy server, preserving both low latency and high availability.

3. Advantages and Limitations of the Proposed Architecture

Dimension	Advantages	Limitations
Performance	Anycast + caching + pre-fetching bring lookup latency down to single-digit/low-double-digit milliseconds for most users.	Pre-fetching can waste resources by resolving domains that are never actually used.
Scalability	Anycast naturally distributes load across many PoPs, allowing the system to absorb traffic spikes (e.g., product launches) without a single point of overload.	Requires BGP routing expertise and coordination with multiple network providers/ISPs to announce the Anycast prefix correctly.
Availability	Automatic failover: if one Anycast node fails, BGP reconvergence routes traffic to the next-nearest healthy node with minimal disruption.	Short TTLs (needed for fast failover) reduce caching efficiency, creating a trade-off between availability and lookup speed.
Consistency	Layered caching improves average response time significantly.	Stale cached records (if TTL is too long) may briefly return outdated IPs during a genuine failover event, causing temporary unreachability for some users.

Overall, the proposed architecture achieves a strong balance: Anycast and caching drive latency well below the 50 ms target for the majority of global users, while redundancy and health-checked failover preserve availability — at the cost of added operational complexity and a careful TTL trade-off between cache efficiency and failover responsiveness.

Question 3: Distributed DNS Solution for a Global Cloud Service Provider

Case Recap

A cloud provider serving millions of users experiences ~800 ms DNS query latency because all requests are routed to a single primary DNS server. A highly available, globally efficient DNS solution is required, especially during high-traffic promotional events.

1. Impact of Centralized DNS Architecture on Application Performance

- Single point of failure: if the one primary DNS server becomes unreachable or overloaded, resolution fails for all users worldwide, causing a complete service outage even though the actual application servers may be healthy.
- Geographic latency penalty: users far from the single server's physical location incur long propagation delays on every query, which is consistent with the reported ~800 ms lookup time.
- Poor scalability under load: during promotional events, the surge in simultaneous DNS queries overwhelms a single server's processing/connection capacity, causing queuing delay and intermittent timeouts — exactly the 'intermittent delays' described in the case.
- No graceful degradation: centralized architectures cannot shed load to nearby healthy nodes, so performance degrades sharply (rather than gradually) once capacity is exceeded.
- Cascading effect on application performance: since DNS resolution is typically the first step of every user interaction, elevated DNS latency directly inflates overall page-load and API response times, harming user experience and potentially revenue during peak promotional traffic.

2. Design of a Distributed DNS Solution

A distributed DNS solution for global scale should include the following components:

1. Anycast-based authoritative DNS network: deploy the same DNS IP address across many geographically distributed data centres/PoPs so that BGP routes each query to the nearest available node, eliminating the single-server bottleneck.

2. Secondary/replica DNS servers: maintain multiple synchronized authoritative servers (primary–secondary replication via zone transfers) across regions so no single node's failure disrupts resolution.
3. Global load-balancing DNS (GSLB): use latency-based or geo-based routing so that, beyond just resolving the name, the DNS answer itself directs users to the nearest healthy application data centre.
4. Multi-layer caching hierarchy: ISP resolvers, CDN edge resolvers, and client-side OS caches absorb the majority of repeat queries, sharply reducing the number of queries that ever reach the authoritative tier.
5. Health-checked automatic failover: continuous health monitoring of each DNS node with automatic withdrawal of unhealthy nodes from Anycast announcements, ensuring queries are never routed to a failed server.
6. Elastic auto-scaling of DNS server capacity during predictable high-traffic events (e.g., promotions), provisioning additional resolver capacity in advance.

3. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

- TTL values: Shorter TTLs allow faster propagation of changes (e.g., quickly rerouting traffic away from a failed data centre) but increase load on authoritative servers and raise average latency because caches expire more often. Longer TTLs reduce load and latency through better caching but slow down failover. A tiered approach — moderate TTLs for stable records and short TTLs only for records tied to load-balancing/failover — balances the two goals.
- Anycast routing: Improves both performance (queries are answered by the nearest node, cutting round-trip latency) and fault tolerance (if a node fails, BGP automatically reconverges to the next-nearest node within seconds, without any client reconfiguration).
- DNS caching: Reduces the number of queries reaching authoritative servers, lowering average latency and server load; however, aggressive caching combined with long TTLs can delay users from seeing a failover update, temporarily reducing effective fault tolerance for a small fraction of clients.

Used together, these three mechanisms allow the platform to serve the vast majority of DNS queries from nearby caches in single-digit milliseconds, while still being able to detect and route around failures within a bounded, configurable time window — directly addressing both the latency and availability goals stated in the case.

Question 4: TCP Latency in a High-Frequency Stock Trading Platform

Case Recap

An electronic trading platform's average order acknowledgment latency rises from 1 ms to 40 ms during market opening hours, with increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Three TCP-Related Factors Responsible for the Latency Increase

a) Flow Control

TCP flow control uses the receive window (rwnd) to prevent a sender from overwhelming a receiver's buffer. During market opening, the sudden burst of orders can fill the receiver's buffer faster than the application can process it, shrinking the advertised window. Once the window closes, the sender must pause and wait for a window update, directly adding latency to order transmission and acknowledgment.

b) Nagle's Algorithm

Nagle's algorithm delays sending small segments until either a full-sized segment can be sent or a pending acknowledgment is received, in order to reduce the overhead of many small packets. Trading messages (orders) are typically small, latency-sensitive packets. If Nagle's algorithm is enabled (the default in most TCP stacks) and is not disabled with TCP_NODELAY, small order packets can be held back waiting to be coalesced or waiting for an ACK, introducing delays of tens of milliseconds — matching the reported jump from 1 ms to 40 ms.

c) SYN Queue Limitations

Each new TCP connection begins with a three-way handshake, and incoming SYN requests are held in a fixed-size SYN queue (backlog) on the server until the handshake completes. During the burst of connection attempts at market open, if the SYN queue is undersized, incoming SYN requests are dropped or delayed, forcing clients to retransmit SYNs after a timeout —

substantially increasing connection establishment time and, consequently, the time before the first order can even be acknowledged.

2. How Each Factor Affects Transaction Processing in High-Frequency Systems

- Flow control: A shrinking receive window throttles the rate at which order messages can be sent, causing orders to queue at the sender; in a high-frequency system this directly delays trade submission and acknowledgment, risking missed price opportunities.
- Nagle's algorithm: By deliberately delaying small packets, it conflicts directly with the low-latency requirement of trading systems, where every millisecond affects execution price — the delay can cause an order to be filled at a worse price or rejected due to a stale quote.
- SYN queue limitations: Slower or failed connection establishment during the highest-value trading window (market open) means some clients cannot even begin submitting orders promptly, leading to missed trading opportunities and potential regulatory/compliance concerns around fair access.

3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm by enabling TCP_NODELAY on trading connections so small order/acknowledgment packets are sent immediately rather than delayed for coalescing.
- Tune TCP window sizes and enable window scaling (RFC 1323) to allow larger receive windows, and ensure the receiving application reads data promptly to keep the window open.
- Increase the SYN backlog queue size (e.g., Linux net.core.somaxconn and net.ipv4.tcp_max_syn_backlog) and enable SYN cookies to handle connection bursts at market open without dropping handshake requests.
- Enable TCP Fast Open (TFO) where supported, to reduce handshake latency for repeat connections between known trading clients and the exchange server.
- Reduce or disable unnecessary application-level and OS-level buffering (excessive buffering) on the critical order path, and consider using a low-latency transport tuned for the exchange (e.g., kernel-bypass networking or UDP-based multicast market-data

feeds combined with a lightweight reliable order-entry protocol) for the most latency-critical paths.

- Monitor and tune the retransmission timeout (RTO) parameters and congestion control algorithm to react faster and avoid unnecessarily conservative backoff during transient congestion at market open.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand